

ENTERPRISE KUBERNETES MASTERY

AI Platform Engineering Handbook

PART XI KUBERNETES FOR AI INFRASTRUCTURE

GPU Operator, KServe, vLLM, Ray, Kubeflow, Distributed Training

Volume 11 of 16 AI Series

Prerequisites: Parts I through X

Edition 2025-2026

TABLE OF CONTENTS

- 1. AI Infrastructure Architecture on Kubernetes 3
- 2. NVIDIA GPU Operator 8
- 3. GPU Scheduling: MIG, Time-Slicing, and Multi-Instance . 13
- 4. Distributed Training: PyTorch DDP and FSDP 17
- 5. Kubeflow: MLOps Platform 22
- 6. Ray and Ray Serve: Distributed AI Compute 27
- 7. KServe: Model Serving Framework 32
- 8. vLLM: High-Throughput LLM Inference 37
- 9. SGLang: Structured Generation at Scale 43
- 10. AI Gateways and Model Routing 46
- 11. Feature Stores with Feast 51
- 12. MLflow: Experiment Tracking and Model Registry . 54
- 13. Vector Databases on Kubernetes 57
- 14. AMD ROCm and Intel Gaudi Support 61
- 15. AI Infrastructure Cost Optimisation 64
- 16. AI Infrastructure Anti-Patterns 68
- 17. Hands-On Exercises 71

CHAPTER 1

AI Infrastructure Architecture on Kubernetes

Kubernetes has become the de facto substrate for enterprise AI infrastructure. It provides the resource scheduling, workload management, and operational tooling that AI/ML workloads require. This part focuses exclusively on Kubernetes implementation patterns for AI infrastructure -- not AI concepts themselves.

AI Workload Taxonomy

Workload Type	Duration	Resource Pattern	K8s Resource	Restart Policy
LLM Inference Serving	Persistent (24x7)	Steady GPU + high RAM	Deployment + HPA/KEDA	Always
Batch Inference	Hours to days	High GPU burst	Job (Indexed)	OnFailure
Distributed Training (LLM)	Days to weeks	All GPUs, all nodes	Volcano Job / PyTorchJob	OnFailure
Fine-tuning (LoRA/QLoRA)	Hours to days	1-8 GPUs	Job or Volcano Job	OnFailure
Hyperparameter Search	Hours	Many parallel small GPU jobs	Kubeflow Katib	OnFailure
Data preprocessing	Hours	CPU + large memory	Job	OnFailure
Embedding generation	Hours	GPU burst	Job	OnFailure
Notebook experiments	Interactive	Shared GPU (MIG)	StatefulSet (JupyterHub)	Always
Pipeline orchestration	Periodic or triggered	Minimal (orchestrator only)	CronJob or Argo WF	OnFailure

Reference AI Platform Architecture

AI Platform on Kubernetes -- layered architecture: LAYER 6: USER INTERFACES Backstage AI Catalogue, JupyterHub, MLflow UI, Kubeflow Pipelines UI, Grafana AI Dashboards LAYER 5: AI ORCHESTRATION Kubeflow Pipelines, Argo Workflows, Ray, Feast Feature Store, MLflow Tracking LAYER 4: MODEL SERVING KServe (multi-framework), vLLM (LLM-optimised), Ray Serve (distributed), Triton Inference Server LAYER 3: AI GATEWAY LiteLLM, OpenRouter, Kong AI Gateway, Model routing, rate limiting, cost tracking, auth LAYER 2: COMPUTE SCHEDULING NVIDIA GPU Operator, Volcano (gang scheduling), Karpenter (GPU node provisioning), KEDA (inference scaling) LAYER 1: KUBERNETES FOUNDATION EKS/GKE/AKS + GPU node pools, Cilium (networking), Ceph/WekaFS (storage), Vault (secrets), ArgoCD (GitOps)

CHAPTER 2

NVIDIA GPU Operator

The NVIDIA GPU Operator automates the deployment and lifecycle management of all NVIDIA software components required to use GPUs in Kubernetes: drivers, container toolkit, device plugin, DCGM exporter, MIG manager, and GPU Feature Discovery. Without the GPU Operator, each component must be manually installed and maintained on every GPU node.

GPU Operator Components

Component	What It Installs	DaemonSet/Operator
Driver container	NVIDIA GPU driver on the host OS	DaemonSet (driver)
Container Toolkit	nvidia-container-runtime; enables GPU in containers	DaemonSet (toolkit)
Device Plugin	Exposes nvidia.com/gpu as schedulable resource	DaemonSet (device-plugin)
DCGM Exporter	GPU telemetry for Prometheus	DaemonSet (dcgm-exporter)
GPU Feature Discovery	Detects GPU capabilities; adds node labels	DaemonSet (gfd)
MIG Manager	Manages MIG partitions on A100/H100	DaemonSet (mig-manager)
Node Feature Discovery	General hardware feature labelling	DaemonSet (nfd)
Validator	Validates GPU stack is working	Pod (validator)

GPU Operator Installation

```
# Add NVIDIA Helm repo: helm repo add nvidia https://helm.ngc.nvidia.com/nvidia helm repo
update # Install GPU Operator: helm install gpu-operator nvidia/gpu-operator \ --namespace
gpu-operator --create-namespace \ --set driver.enabled=true \ --set driver.version=550.54.15 \
--set toolkit.enabled=true \ --set devicePlugin.enabled=true \ --set dcgmExporter.enabled=true
\ --set gfd.enabled=true \ --set mig.strategy=single \ --set
validator.plugin.env[0].name=WITH_WORKLOAD \ --set validator.plugin.env[0].value='true' #
Verify GPU is available: kubectl get nodes -l nvidia.com/gpu.present=true kubectl describe
node GPU_NODE | grep nvidia.com # Should show: nvidia.com/gpu: 8 (for 8-GPU node) # GPU node
labels added by GPU Feature Discovery: # nvidia.com/gpu.product=NVIDIA-A100-SXM4-80GB #
nvidia.com/gpu.memory=81920 (80GB in MiB) # nvidia.com/gpu.count=8 #
nvidia.com/gpu.family=ampere
```

GPU Node Taint Strategy

```
# Taint GPU nodes to prevent non-GPU workloads consuming them: kubectl taint nodes -l
nvidia.com/gpu.present=true \ nvidia.com/gpu=true:NoSchedule # GPU workload tolerates the
taint: spec: tolerations: - key: nvidia.com/gpu operator: Exists effect: NoSchedule
containers: - resources: limits: nvidia.com/gpu: '1' # NodeSelector for specific GPU model:
nodeSelector: nvidia.com/gpu.product: NVIDIA-H100-SXM5-80GB
```

CHAPTER 3

GPU Scheduling: MIG, Time-Slicing, and Multi-Instance

Full GPU allocation (one GPU per container) is often wasteful for small models or development workloads. NVIDIA provides three approaches to share GPU resources among multiple workloads, each with different isolation and performance characteristics.

GPU Sharing Approaches

Approach	Isolation	Memory	Performance	Best For
Full GPU	Complete	Dedicated 100%	100%	LLM inference, large training
MIG (Multi-Instance GPU)	Hardware	Dedicated slice	Proportional to slice	Parallel small inference, dev notebooks
Time-slicing	Temporal only	Shared (no isolation)	Variable (context switch)	Dev workloads, testing
MPS (Multi-Process Service)	Process-level	Shared	High for compatible workloads	Batch inference, same-model multi-tenant

NVIDIA MIG (Multi-Instance GPU)

MIG partitions an A100 or H100 GPU into up to 7 hardware-isolated instances. Each instance has dedicated streaming multiprocessors (SMs), memory bandwidth, and L2 cache. Unlike time-slicing, MIG instances cannot interfere with each other.

```
# A100 80GB MIG profiles: # 1g.10gb = 1/7 GPU, 10GB VRAM (7 instances max) # 2g.20gb = 2/7 GPU, 20GB VRAM (3 instances max) # 3g.40gb = 3/7 GPU, 40GB VRAM (2 instances max) # 4g.40gb = 4/7 GPU, 40GB VRAM (1 instance) # 7g.80gb = full GPU, 80GB VRAM (1 instance, MIG disabled) # H100 SXM5 80GB MIG profiles: # 1g.10gb, 2g.20gb, 3g.40gb, 4g.40gb, 7g.80gb (same pattern) # Enable MIG mode on a node (GPU Operator MIG Manager): kubectl label node GPU_NODE nvidia.com/mig.config=all-1g.10gb # MIG Manager DaemonSet sees the label, configures MIG on that node # After MIG is enabled, resources appear as: kubectl describe node GPU_NODE | grep mig # nvidia.com/mig-1g.10gb: 7 # Request a MIG instance in a Pod: resources: limits: nvidia.com/mig-1g.10gb: '1'
```

CHAPTER 4

Distributed Training: PyTorch DDP and FSDP

Training large language models requires distributing compute across multiple GPUs and often multiple nodes. Kubernetes provides the infrastructure substrate; PyTorch provides the distributed training frameworks. Understanding the parallelism strategies is essential for designing efficient training infrastructure.

Parallelism Strategies

Strategy	Distributes	When to Use	Memory Reduction	K8s Pattern
Data Parallel (DDP)	Data batches across GPUs	Model fits in single GPU	None (model replicated)	Job with NCCL all-reduce
Fully Sharded Data Parallel (FSDP)	Model parameters + gradients + optimizer	70B+ models, limited VRAM	Up to 8x	Job with NVLink/InfiniBand
Tensor Parallel (TP)	Individual tensors across GPUs	Attention layers too large for 1 GPU	Proportional to TP degree	PyTorchJob, same-node GPUs
Pipeline Parallel (PP)	Model layers across nodes	Very deep models	Proportional to PP degree	PyTorchJob, multi-node
Sequence Parallel	Sequence dimension of attention	Very long context windows	Proportional	Advanced, with Megatron-LM

PyTorchJob with Kubeflow Training Operator

```
# Distributed PyTorch training on 4 nodes x 8 GPUs = 32 GPUs total: apiVersion:
kubeflow.org/v1 kind: PyTorchJob metadata: name: llm-finetune-llama3-70b namespace:
ml-training spec: pytorchReplicaSpecs: Master: replicas: 1 restartPolicy: OnFailure template:
spec: tolerations: - key: nvidia.com/gpu operator: Exists effect: NoSchedule containers: -
name: trainer image: harbor.corp/llm-trainer:v2.0 command: - torchrun - --nproc_per_node=8 -
--nnodes=4 - --node_rank=$(RANK) - --master_addr=$(MASTER_ADDR) - --master_port=23456 -
train_fsdp.py - --model=meta-llama/Llama-3-70b - --dataset=s3://training-data/dataset-v3 -
--output=s3://checkpoints/llama3-70b-ft-v1 resources: limits: nvidia.com/gpu: '8' memory:
1500Gi cpu: '192' requests: nvidia.com/gpu: '8' memory: 1500Gi env: - name: NCCL_DEBUG value:
INFO - name: NCCL_SOCKET_IFNAME value: eth0 - name: NCCL_IB_DISABLE value: '0' # Enable
InfiniBand Worker: replicas: 3 # 3 workers + 1 master = 4 nodes total
```


CHAPTER 5

Kubeflow: MLOps Platform

Kubeflow is a collection of Kubernetes-native ML tools that together form an end-to-end MLOps platform. It provides pipeline orchestration, distributed training, hyperparameter optimisation, model serving, and notebook management on Kubernetes.

Kubeflow Components

Component	Function	CRD / API
Kubeflow Pipelines (KFP)	DAG-based ML workflow orchestration; tracks runs, artifacts	Pipeline, PipelineRun
Training Operator	Manages distributed training jobs	PyTorchJob, TFJob, MXJob, JAXJob, PaddleJob
Katib	Automated hyperparameter search (Bayesian, random, grid)	Experiment, Trial
KServe	Model serving (InferenceService)	InferenceService
Notebooks	JupyterHub-based notebook server management	Notebook
Volumes	PVC management for notebooks and pipelines	PodDefault

Kubeflow Pipeline Example

```
# KFP v2 Python SDK pipeline: from kfp import dsl, compiler from kfp.dsl import component, pipeline, Dataset, Model @component(base_image='python:3.12', packages_to_install=['pandas','scikit-learn']) def preprocess_data(input_path: str, output_dataset: dsl.Output[Dataset]): import pandas as pd df = pd.read_parquet(input_path) df_clean = df.dropna() df_clean.to_parquet(output_dataset.path) @component(base_image='nvcr.io/nvidia/pytorch:24.05-py3', resources=dsl.ResourceSpec(accelerator_type='NVIDIA_TESLA_A100', accelerator_count=4)) def finetune_model(dataset: dsl.Input[Dataset], output_model: dsl.Output[Model]): # LoRA fine-tuning code here pass @component(base_image='python:3.12', packages_to_install=['mlflow']) def register_model(model: dsl.Input[Model], experiment_name: str): import mlflow mlflow.log_artifacts(model.path) @pipeline(name='llm-finetune-pipeline', description='LLM fine-tuning with LoRA') def llm_pipeline(input_path: str = 's3://data/training-v3', experiment: str = 'llama-finetune-v1'): preprocess_task = preprocess_data(input_path=input_path) finetune_task = finetune_model(dataset=preprocess_task.outputs['output_dataset']) register_task = register_model( model=finetune_task.outputs['output_model'], experiment_name=experiment ) compiler.Compiler().compile(llm_pipeline, 'llm-pipeline.yaml')
```

CHAPTER 6

Ray and Ray Serve: Distributed AI Compute

Ray is a distributed computing framework purpose-built for AI workloads. It enables scaling Python code from a laptop to a cluster without rewriting, provides native support for GPU scheduling, and includes Ray Serve for production model serving. The KubeRay Operator manages Ray clusters on Kubernetes.

Ray Architecture on Kubernetes

```
KubeRay Operator manages RayCluster CRDs: RayCluster: Head Node (Deployment): Ray head process (scheduler, GCS, dashboard) Typically CPU-only, medium memory Worker Groups (Deployment or StatefulSet): Multiple worker groups with different resource profiles Example: cpu-workers, gpu-workers-a100, gpu-workers-h100 RayJob: Creates a temporary RayCluster for a single job Deletes cluster when job completes Ideal for batch inference and training jobs RayService: Long-running Ray Serve deployment Manages: head + workers + Serve deployment config Zero-downtime upgrades via in-place rolling updates
```

RayCluster for LLM Inference

```
apiVersion: ray.io/v1 kind: RayCluster metadata: name: llm-serving-cluster namespace: ai-serving spec: headGroupSpec: replicas: 1 template: spec: containers: - name: ray-head image: rayproject/ray-ml:2.20.0-py311-gpu resources: requests: { cpu: '8', memory: 32Gi } limits: { cpu: '8', memory: 32Gi } ports: - containerPort: 6379 # GCS port - containerPort: 8265 # Dashboard - containerPort: 10001 # Client workerGroupSpecs: - groupName: gpu-workers replicas: 4 minReplicas: 2 maxReplicas: 8 template: spec: tolerations: - key: nvidia.com/gpu operator: Exists effect: NoSchedule containers: - name: ray-worker image: rayproject/ray-ml:2.20.0-py311-gpu resources: requests: { cpu: '32', memory: 256Gi, nvidia.com/gpu: '4' } limits: { cpu: '32', memory: 256Gi, nvidia.com/gpu: '4' }
```

CHAPTER 7

KServe: Model Serving Framework

KServe (formerly KFServing, CNCF incubating) is the standard Kubernetes-native model serving framework. It abstracts model serving complexity behind a single InferenceService CRD, supporting multiple frameworks (PyTorch, TensorFlow, sklearn, XGBoost, Triton) with auto-scaling, canary deployments, and request batching.

KServe Architecture

```
KServe components: InferenceService controller: Reconciles InferenceService CRD Creates:
Deployment (predictor), Service, Istio VirtualService Predictor: Runs model serving runtime
(Triton, vLLM, Hugging Face TGI) Loads model from storage (S3, GCS, PVC) Exposes: gRPC (port
9000) and HTTP (port 8080) Transformer (optional): Pre/post processing before/after predictor
Custom Python code for input transformation Explainer (optional): Model explanation endpoints
(SHAP, LIME, Captum) Knative Serving: Provides scale-to-zero and request-based autoscaling
KServe uses Knative for auto-scaling out of the box
```

InferenceService for LLM (vLLM runtime)

```
apiVersion: serving.kserve.io/v1beta1 kind: InferenceService metadata: name:
llama3-70b-instruct namespace: ai-serving annotations: serving.kserve.io/deploymentMode:
RawDeployment spec: predictor: model: modelFormat: name: huggingface runtime: kserve-vllm
storageUri: s3://company-models/llama3-70b-instruct args: - --tensor-parallel-size=4 -
--max-model-len=8192 - --enable-prefix-caching - --dtype=bfloat16 resources: requests:
nvidia.com/gpu: '4' memory: 256Gi cpu: '16' limits: nvidia.com/gpu: '4' memory: 256Gi
minReplicas: 1 maxReplicas: 4 scaleMetric: rps # Requests per second scaleTarget: 10 # Scale
up when > 10 RPS per replica # Query the InferenceService: curl -X POST \
http://llama3-70b-instruct.ai-serving.svc.cluster.local/v1/chat/completions \ -H
'Content-Type: application/json' \ -d
'{"model":"llama3-70b","messages":[{"role":"user","content":"Hello"}]}'
```

CHAPTER 8

vLLM: High-Throughput LLM Inference

vLLM is the leading open-source LLM inference engine for production deployments. It achieves 3-24x higher throughput than naive HuggingFace Transformers inference through PagedAttention (GPU memory management), continuous batching, and prefix caching. It is OpenAI-compatible and runs on Kubernetes natively.

vLLM Key Innovations

- * **PagedAttention:** Traditional attention allocates GPU memory for maximum sequence length upfront, wasting memory on shorter sequences. PagedAttention manages KV cache memory in non-contiguous pages (like OS virtual memory), enabling near-100% GPU memory utilisation and 3-5x more concurrent requests per GPU.
- * **Continuous Batching:** Instead of waiting for a batch to complete, new requests are added to running batches as sequences finish. This eliminates GPU idle time between batches and dramatically increases throughput for mixed-length workloads.
- * **Prefix Caching:** Common prompt prefixes (system prompts, few-shot examples) are cached in GPU memory and reused across requests. For RAG workloads with long context, prefix caching reduces TTFT by 50-80%.
- * **Tensor Parallelism:** Model weights are sharded across multiple GPUs using tensor parallelism. A 70B model requiring 140GB VRAM can run across 2x 80GB GPUs (2-way tensor parallel) or 4x 40GB GPUs (4-way tensor parallel).

vLLM Production Kubernetes Deployment

```
apiVersion: apps/v1 kind: Deployment metadata: name: vllm-llama3-70b namespace: ai-serving
spec: replicas: 2 selector: matchLabels: { app: vllm-llama3-70b } template: spec: tolerations:
- key: nvidia.com/gpu operator: Exists effect: NoSchedule containers: - name: vllm image:
vllm/vllm-openai:v0.5.0 command: [python, -m, vllm.entrypoints.openai.api_server] args: -
--model=/models/llama3-70b-instruct - --tensor-parallel-size=4 - --max-model-len=8192 -
--enable-prefix-caching - --dtype=bfloat16 - --gpu-memory-utilization=0.92 -
--max-num-batched-tokens=32768 - --port=8000 - --host=0.0.0.0 -
--served-model-name=llama3-70b-instruct resources: limits: { nvidia.com/gpu: '4', memory:
512Gi, cpu: '64' } requests: { nvidia.com/gpu: '4', memory: 512Gi, cpu: '64' } volumeMounts: -
name: model-weights mountPath: /models readOnly: true readinessProbe: httpGet: { path:
/health, port: 8000 } initialDelaySeconds: 120 # Model loading takes time periodSeconds: 10
startupProbe: httpGet: { path: /health, port: 8000 } failureThreshold: 60 # Up to 10 minutes
for 70B model load periodSeconds: 10 volumes: - name: model-weights persistentVolumeClaim:
claimName: llama3-70b-weights
```

CHAPTER 9

SGLang: Structured Generation at Scale

SGLang (Structured Generation Language) is an LLM inference framework optimised for structured outputs and multi-call programmes. It provides RadixAttention (a more efficient variant of prefix caching) and is particularly well-suited for agentic workloads that involve structured JSON outputs, multi-step reasoning, and repeated calls with shared prefixes.

SGLang vs vLLM Comparison

Feature	vLLM	SGLang
Architecture	PagedAttention + continuous batching	RadixAttention + compressed FSM
Structured output	Via guided generation (slow)	Native compressed finite-state machine (fast)
Prefix caching	Block-level prefix cache	Radix tree (more granular sharing)
Multi-call programs	Not native	Native via SGL language primitives
JSON schema enforcement	Outlines integration	Built-in, 10x faster than vLLM+outlines
DP Attention	Partial	Full data parallelism in disaggregated serving
Best for	General LLM serving, high throughput	Agentic AI, structured output, tool calling

SGLang Kubernetes Deployment

```
# SGLang server deployment: containers: - name: sglang image: lmsysorg/sglang:latest-cu121
command: - python - -m - sglang.launch_server args: -
--model-path=/models/qwen2.5-72b-instruct - --tp=4 - --port=30000 - --host=0.0.0.0 -
--enable-torch-compile # Torch compile for 20% speedup - --max-prefill-tokens=16384 resources:
limits: { nvidia.com/gpu: '4', memory: 512Gi }
```

CHAPTER 10

AI Gateways and Model Routing

An AI Gateway sits between client applications and LLM backends, providing: unified API surface across multiple models, authentication and authorisation, rate limiting and cost controls, request routing (model selection), caching, observability, and fallback handling. As LLM usage scales, an AI gateway becomes essential infrastructure.

AI Gateway Capabilities

Capability	Business Value	Implementation
Unified API	Apps use one endpoint regardless of backend model	OpenAI-compatible /v1 endpoint for all models
Model routing	Route to best model by cost/latency/capability	Rules-based or ML-based routing
Authentication	Secure model access; track usage per team/user	API key or OIDC JWT validation
Rate limiting	Prevent runaway costs; enforce quotas per team	Token-per-minute limits per key
Semantic caching	Cache responses for identical/similar queries	Embedding similarity threshold cache
Cost tracking	Allocate AI costs to teams/applications	Per-request token tracking -> OpenCost
Fallback handling	Route to backup model on primary failure	Retry with fallback model list
PII redaction	Strip sensitive data before sending to external LLMs	Presidio or custom regex pipeline
Prompt injection detection	Detect and block adversarial prompts	LLM-based or pattern-based detection

LiteLLM Proxy on Kubernetes

```
# LiteLLM Proxy -- unified AI gateway: apiVersion: apps/v1 kind: Deployment metadata: name:
litellm-proxy namespace: ai-gateway spec: replicas: 3 template: spec: containers: - name:
litellm image: ghcr.io/berriai/litellm:main-latest args: ['--config', '/app/config.yaml',
'--port', '4000'] resources: requests: { cpu: 1, memory: 2Gi } limits: { memory: 4Gi }
volumeMounts: - name: config mountPath: /app/config.yaml subPath: config.yaml # litellm
config.yaml: model_list: - model_name: gpt-4o litellm_params: model: azure/gpt-4o api_base:
https://company.openai.azure.com api_key: os.environ/AZURE_API_KEY - model_name: llama3-70b
```

```
litellm_params: model: openai/llama3-70b-instruct api_base:  
http://vllm-llama3-70b.ai-serving:8000 api_key: dummy - model_name: claude-3-5-sonnet  
litellm_params: model: anthropic/claude-3-5-sonnet-20241022 api_key:  
os.environ/ANTHROPIC_API_KEY router_settings: routing_strategy: least-busy num_retries: 3  
fallbacks: [{gpt-4o: [llama3-70b]}] general_settings: master_key:  
os.environ/LITELLM_MASTER_KEY database_url: postgresql://litellm:PASS@postgres:5432/litellm
```

CHAPTER 11

Feature Stores with Feast

A feature store is the data layer that bridges data engineering and machine learning. It provides consistent, versioned, low-latency feature serving for both training (batch) and inference (online). Feast is the most widely deployed open-source feature store, with native Kubernetes integration.

Feast Architecture on Kubernetes

```
Feast components: Offline Store (batch features for training): Reads from: BigQuery, Redshift,
Snowflake, Spark, DuckDB Provides historical feature values for training dataset generation
Online Store (real-time features for inference): Stores in: Redis, DynamoDB, Bigtable, SQLite
(dev) Provides sub-millisecond feature lookup during inference Feature Server (HTTP API for
online features): REST endpoint: GET /get-online-features Deployed as Kubernetes Deployment
Materialisation Job: Batch job that copies offline -> online store Scheduled via Kubernetes
CronJob Feature Registry: Stores feature definitions (GCS, S3, SQL database) # Feast feature
definition: from feast import Entity, Feature, FeatureView, ValueType from feast.types import
Float64, Int64 user = Entity(name='user_id', value_type=ValueType.STRING) user_features =
FeatureView( name='user_features', entities=[user], ttl=timedelta(days=30), schema=[
Field(name='age', dtype=Int64), Field(name='lifetime_value', dtype=Float64),
Field(name='preferred_language', dtype=String), ],
source=BigQuerySource(table='company.features.user_features') )
```


CHAPTER 12

MLflow: Experiment Tracking and Model Registry

MLflow is the leading open-source MLOps platform for experiment tracking, model packaging, and model registry. In Kubernetes, MLflow serves as the system of record for all training runs, model versions, and deployment history.

MLflow Components on Kubernetes

```
MLflow deployment architecture: MLflow Tracking Server (Deployment): REST API: log metrics,
parameters, artifacts Backend: PostgreSQL (runs, experiments, params, metrics) Artifact store:
S3, GCS, Azure Blob (model files, plots) MLflow Model Registry: Versioned model storage
Lifecycle stages: Staging -> Production -> Archived Approval workflow via REST API or MLflow
UI Model serving via MLflow: mlflow models serve: built-in serving (dev only) Production:
export to KServe, vLLM, or custom serving # Training code integration: import mlflow
mlflow.set_tracking_uri('http://mlflow.mlops.svc.cluster.local:5000')
mlflow.set_experiment('llama3-finetune-v2') with mlflow.start_run(): mlflow.log_param('model',
'meta-llama/Llama-3-70b') mlflow.log_param('learning_rate', 2e-4)
mlflow.log_param('num_epochs', 3) for epoch in range(num_epochs): train_loss =
train_one_epoch() mlflow.log_metric('train_loss', train_loss, step=epoch)
mlflow.pytorch.log_model( model, 'model', registered_model_name='llama3-70b-finetune-v2' )
```

CHAPTER 13

Vector Databases on Kubernetes

Vector databases are foundational infrastructure for RAG pipelines. They store high-dimensional embeddings and enable approximate nearest neighbour (ANN) search to find semantically similar content. Production deployments require careful sizing, scaling, and operational management on Kubernetes.

Vector Database Comparison

Database	Algorithm	K8s Native	Scale-Out	Best For
Qdrant	HNSW (disk-indexed)	StatefulSet + Operator	Sharded cluster	Enterprise RAG, on-prem
Weaviate	HNSW (in-memory)	StatefulSet + Operator	Horizontal (modules)	Complex filtering + search
Milvus	Multiple (HNSW, IVF, FLAT)	Operator (Milvus Operator)	Fully distributed	Large scale, enterprise
pgvector	IVFFlat / HNSW	StatefulSet (CloudNativePG)	PostgreSQL HA	Existing Postgres users
Chroma	HNSW (in-process)	Deployment	Single server	Prototyping, small scale
Redis (RedisSearch)	HNSW + inverted index	StatefulSet	Redis Cluster	Low latency, hybrid search

Milvus Operator Deployment

```
# Install Milvus Operator: helm repo add milvus-operator
https://zilliztech.github.io/milvus-operator/ helm install milvus-operator
milvus-operator/milvus-operator \ --namespace milvus-operator --create-namespace # Create a
Milvus cluster: apiVersion: milvus.io/v1beta1 kind: Milvus metadata: name: milvus-prod
namespace: ai-platform spec: mode: cluster components: queryNode: replicas: 3 resources:
requests: { cpu: 8, memory: 64Gi } indexNode: replicas: 2 resources: requests: { cpu: 16,
memory: 32Gi } dataNode: replicas: 2 dependencies: etcd: { inCluster: { values: {
replicaCount: 3 } } } minio: { inCluster: { values: { mode: distributed } } } pulsar: {
inCluster: { enabled: true } }
```

CHAPTER 14

AMD ROCm and Intel Gaudi Support

NVIDIA dominates AI accelerator mindshare, but AMD MI300X and Intel Gaudi 3 offer compelling alternatives with significant cost advantages. Kubernetes supports all three via device plugins and operator patterns.

Accelerator Comparison

Accelerator	VRAM	FP16 TFLOPS	Int8 TOPS	K8s Integration	Key Use Case
NVIDIA H100 SXM5	80GB HBM3	1,979	3,958	GPU Operator + CUDA	Best ecosystem, LLM training
NVIDIA A100 SXM4	80GB HBM2e	312	624	GPU Operator + CUDA	Training, inference baseline
AMD MI300X	192GB HBM3	1,307	2,614	ROCm Device Plugin	Large models (192GB = 70B+ no sharding)
Intel Gaudi 3	96GB HBM2e	1,835	3,670	Intel Gaudi Operator	Cost-effective training, PyTorch native

AMD ROCm on Kubernetes

```
# AMD ROCm Device Plugin: helm repo add rocm https://rocm.github.io/k8s-device-plugin helm
install rocm-device-plugin rocm/rocm-device-plugin \ --namespace kube-system # AMD GPU appears
as: # amd.com/gpu resource on nodes # Run vLLM on AMD MI300X: containers: - name: vllm-rocm
image: vllm/vllm-rocm:latest command: [python, -m, vllm.entrypoints.openai.api_server] args: -
--model=meta-llama/Llama-3-70B-Instruct - --tensor-parallel-size=1 # MI300X has 192GB -- fits
70B solo! resources: limits: { amd.com/gpu: '1' }
```

CHAPTER 15

AI Infrastructure Cost Optimisation

GPU compute is the dominant cost in enterprise AI. A single H100 GPU node costs over \$30/hour on-demand from AWS. Optimising GPU utilisation, scheduling strategy, and model efficiency can reduce AI infrastructure costs by 50-80%.

Cost Optimisation Strategies

- * **GPU utilisation maximisation:** Batch requests to maximise GPU compute utilisation. vLLM continuous batching enables 70-90% GPU utilisation vs 20-30% for naive serving. Monitor `DCGM_FI_DEV_GPU_UTIL` -- target > 70% during serving hours.
- * **Spot/preemptible instances for training:** Use spot instances (AWS) or preemptible VMs (GCP) for batch training jobs -- 60-90% cheaper than on-demand. Implement checkpoint-based fault tolerance so jobs resume after preemption.
- * **Scale to zero for inference:** Use KEDA to scale inference deployments to zero replicas during off-peak hours. Critical for internal tools with predictable usage patterns. Model loading time (1-10 min for large LLMs) is acceptable for non-latency-sensitive apps.
- * **Reserved/committed use discounts:** Commit to 1 or 3 years of GPU instance usage for 60-70% discount over on-demand. Use for baseline inference capacity; use spot for training peaks.
- * **Quantisation:** INT8 and INT4 quantisation reduces GPU memory requirements by 2-4x with minimal quality loss for most use cases. 70B FP16 model (140GB) becomes 70B INT8 (70GB) -- fits on single 80GB GPU instead of requiring 2.
- * **Model right-sizing:** Deploy the smallest model that meets quality requirements. 8B model costs 10x less to serve than 70B model. Use A/B testing to find the quality/cost tradeoff.
- * **Prefix caching:** Enable vLLM prefix caching for RAG workloads with long context. Cache hit rates of 50-80% reduce compute by proportional amount. Monitor `vllm:gpu_prefix_cache_hit_rate` metric.

CHAPTER 16

AI Infrastructure Anti-Patterns

Anti-Pattern: One model per Deployment for serving

Problem: Running separate Deployments for each LLM variant wastes GPU resources. 3 models x 4 GPUs = 12 GPUs sitting idle between requests.

Solution: Use model multiplexing in vLLM (--model flag with multiple models) or Triton Inference Server for co-locating compatible models on shared GPUs.

Anti-Pattern: Not setting GPU request == limit

Problem: GPU resources in Kubernetes require request == limit. Setting only requests without limits causes incorrect scheduling.

Solution: Always set both: requests.nvidia.com/gpu and limits.nvidia.com/gpu to the same value.

Anti-Pattern: Training without checkpointing

Problem: Multi-day training job fails at 95% completion with no checkpoint. Must restart from scratch. Total loss of GPU hours.

Solution: Checkpoint every N steps to S3/PVC. Implement resume-from-checkpoint in training code. Test checkpoint/resume in dev before long training runs.

Anti-Pattern: LLM inference without batching

Problem: Serving one request at a time on a 4-GPU H100 node. GPU utilisation: 5%. Cost per token is 20x higher than necessary.

Solution: Use vLLM or SGLang with continuous batching enabled. Set appropriate max-num-batched-tokens for your request mix.

Anti-Pattern: No GPU node taints

Problem: General workloads scheduled onto GPU nodes. CPU-only Pods consume GPU node resources, blocking AI workloads from scheduling.

Solution: Always taint GPU nodes: nvidia.com/gpu=true:NoSchedule. Only Pods with explicit GPU requests and tolerations can use GPU nodes.

CHAPTER 17

Hands-On Exercises

Exercise 11.1 -- Deploy vLLM for LLM Inference

Deploy a small LLM model using vLLM on a GPU node:

```
# Prerequisite: GPU node with NVIDIA GPU Operator installed # Deploy vLLM with a small model
(Qwen2.5-1.5B for dev): kubectl apply -f - <<'YAML' apiVersion: apps/v1 kind: Deployment
metadata: name: vllm-dev namespace: ai-dev spec: replicas: 1 selector: matchLabels: { app:
vllm-dev } template: metadata: labels: { app: vllm-dev } spec: tolerations: - key:
nvidia.com/gpu operator: Exists effect: NoSchedule containers: - name: vllm image:
vllm/vllm-openai:latest args: - --model=Qwen/Qwen2.5-1.5B-Instruct - --port=8000 -
--host=0.0.0.0 resources: limits: { nvidia.com/gpu: '1', memory: 16Gi } YAML # Expose and
test: kubectl expose deployment vllm-dev --port=8000 -n ai-dev kubectl port-forward -n ai-dev
svc/vllm-dev 8000:8000 # Test OpenAI-compatible API: curl
http://localhost:8000/v1/chat/completions \ -H 'Content-Type: application/json' \ -d
'{"model": "Qwen2.5-1.5B-Instruct", "messages": [{"role": "user", "content": "Hello!"}]}'
```

Exercise 11.2 -- GPU Utilisation Monitoring

Observe GPU metrics during inference load:

```
# Install DCGM Exporter (if not installed via GPU Operator): helm repo add gpu-helm-charts
https://nvidia.github.io/dcg-exporter/helm-charts helm install dcgm-exporter
gpu-helm-charts/dcg-exporter \ --namespace monitoring # Verify GPU metrics are available in
Prometheus: kubectl port-forward -n monitoring svc/kube-prom-kube-prometheus-prometheus 9090 #
Query: DCGM_FI_DEV_GPU_UTIL # Generate load on vLLM: pip install locust # Create locustfile.py
for LLM load test locust -H http://localhost:8000 --users=10 --spawn-rate=2 # Observe GPU
utilisation rise in Grafana/Prometheus: # DCGM_FI_DEV_GPU_UTIL should rise toward 80-90% #
DCGM_FI_DEV_FB_USED shows KV cache growing
```

End of Part XI -- Continue to Part XII: Kubernetes for Enterprise Agentic AI

Part XII focuses exclusively on Kubernetes implementation patterns for enterprise Agentic AI: stateless vs stateful agent execution, durable workflows with Temporal and Argo Workflows, MCP server deployment patterns, A2A communication across clusters, agent scheduling and autoscaling, tool and prompt registries, AI observability for multi-agent systems, GitOps for AI assets, and multi-region sovereign AI deployments.